Manual Técnico del Desarrollador

Sistema de Login Multicapa · Arquitectura, mantenimiento y despliegue

Versión	1.0
Fecha	15/10/2026
Proyecto	proyecto-login · Arquitectura multicapa J2EE
Ámbito	UF2405 · UD2: Programación en capas

Versión: 1.0

Fecha: 15/10/2026

Autor: Departamento de Desarrollo

Proyecto: proyecto-login - Arquitectura Multicapa J2EE

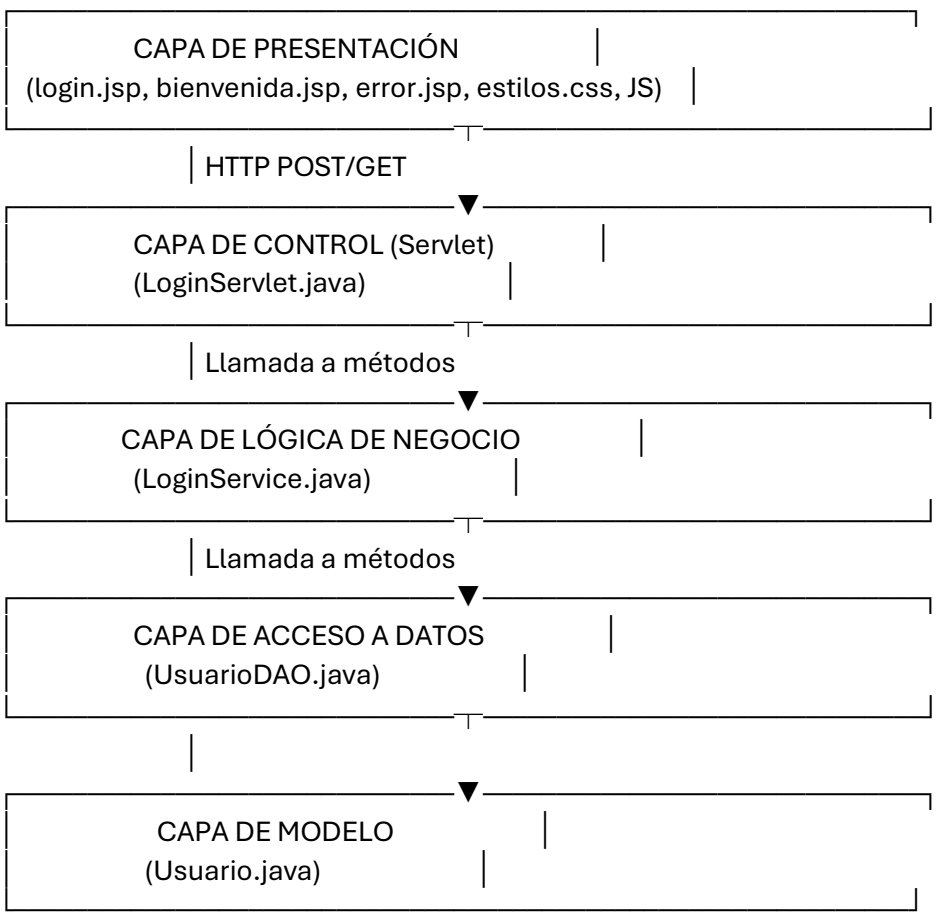
1. Introducción

Este documento está dirigido a **desarrolladores y administradores de sistemas** que necesiten comprender, mantener, extender o desplegar el proyecto proyecto-login. Describe la arquitectura técnica, la estructura del código, los patrones de diseño utilizados y los procedimientos de compilación y despliegue.

2. Arquitectura del Sistema

2.1. Visión General

El sistema sigue una **arquitectura de 3 capas** basada en el patrón **Modelo-Vista-Controlador (MVC)**:



2.2. Tecnologías Utilizadas

Componente	Tecnología	Versión
Lenguaje de programación	Java	JDK 8+
API de Servlets	javax.servlet-api	3.1.0
Servidor de aplicaciones	Apache Tomcat (embebido)	7.0.47
Gestor de dependencias	Apache Maven	3.9.9
Vistas dinámicas	JSP (Java Server Pages)	2.3
Cliente web	HTML5 + CSS3 + JavaScript	ES5
Base de datos	Simulación en memoria (ArrayList)	-

3. Estructura del Proyecto

3.1. Árbol de Directorios

```
proyecto-login/
├── pom.xml                  # Configuración de Maven
├── instalar_maven_local.bat      # Script de instalación local de Maven
├── lanzar_proyecto.bat           # Script de compilación y despliegue
│
└── src/
    └── main/
        ├── java/              # Código fuente Java
        │   └── com/
        │       └── empresa/
        │           └── login/
        │               ├── model/
        │               │   └── Usuario.java        # JavaBean (Entidad)
        │               ├── dao/
        │               │   └── UsuarioDAO.java      # Acceso a datos
        │               ├── service/
        │               │   └── LoginService.java    # Lógica de negocio
        │               └── web/
        │                   └── LoginServlet.java    # Controlador
        │
        └── webapp/              # Recursos web (Capa de presentación)
            ├── WEB-INF/
            │   └── web.xml        # Descriptor de despliegue
            ├── css/
            │   └── estilos.css     # Hoja de estilos
            ├── js/
            │   └── script.js      # Validación cliente
```

```
├── login.jsp          # Página de login
├── bienvenida.jsp     # Página de éxito
└── error.jsp          # Página de error
```

3.2. Descripción de Componentes

3.2.1. `pom.xml`

Archivo de configuración de Maven que define:

GroupId: com.empresa

ArtifactId: proyecto-login

Packaging: war (Web Application Archive)

Dependencias: javax.servlet-api (scope: provided)

Plugins: maven-compiler-plugin y tomcat7-maven-plugin

3.2.2. Capa de Modelo (`model/Usuario.java`)

JavaBean que representa la entidad Usuario. Implementa Serializable para poder almacenarse en la sesión HTTP.

Atributos:

username (String)

password (String)

nombreCompleto (String)

rol (String)

3.2.3. Capa de Acceso a Datos (`dao/UsuarioDAO.java`)

Clase responsable de interactuar con la fuente de datos. Actualmente usa una ArrayList en memoria como simulación.

Métodos principales:

validarCredenciales(String username, String password): boolean

buscarPorUsername(String username): Usuario

obtenerTodosLosUsuarios(): List<Usuario>

3.2.4. Capa de Lógica de Negocio (`service/LoginService.java`)

Clase que encapsula las reglas de negocio. Actúa como intermediario entre el Servlet y el DAO.

Métodos principales:

autenticar(String username, String password): boolean

obtenerDatosUsuario(String username): Usuario

3.2.5. Capa de Control (`web/LoginServlet.java`)

Servlet que actúa como controlador MVC. Recibe las peticiones HTTP, invoca la lógica de negocio y decide qué vista mostrar.

Métodos:

init(): Inicializa el servicio.

doPost(): Procesa el envío del formulario de login.

doGet(): Redirige a login.jsp.

4. Configuración del Entorno de Desarrollo

4.1. Requisitos Previos

Java JDK 8 o superior instalado y accesible desde el PATH.

Apache Maven 3.9.9 (puede instalarse localmente con el script instalar_maven_local.bat).

Editor de código recomendado: Visual Studio Code, IntelliJ IDEA o Eclipse.

Navegador web para pruebas (Chrome, Firefox, Edge).

4.2. Instalación Local de Maven (Sin afectar al SO)

Para mantener el sistema operativo limpio, el proyecto incluye un script que descarga Maven en la carpeta tools/:

instalar_maven_local.bat

Este script:

Verifica que Java esté instalado.

Crea la carpeta tools/.

Descarga Apache Maven 3.9.9 desde el repositorio oficial.

Extrae el contenido en tools/apache-maven-3.9.9/.

4.3. Compilación y Despliegue

Para compilar el proyecto y levantar el servidor Tomcat embebido:

lanzar_proyecto.bat

Este script:

Detecta Java y Maven local.

Configura el PATH temporalmente.

Ejecuta mvn clean compile tomcat7:run.

Levanta el servidor en http://localhost:8080/proyecto-login/.

Para detener el servidor: Presione Ctrl + C en la ventana de la consola.

5. Flujo de una Petición HTTP

5.1. Diagrama de Secuencia (Login Correcto)

```
Usuario -> login.jsp: Introduce credenciales
login.jsp -> LoginServlet: POST /LoginServlet (username, password)
LoginServlet -> LoginService: autenticar(username, password)
LoginService -> UsuarioDAO: validarCredenciales(username, password)
UsuarioDAO --> LoginService: true
LoginService --> LoginServlet: true
LoginServlet -> LoginService: obtenerDatosUsuario(username)
LoginService -> UsuarioDAO: buscarPorUsername(username)
UsuarioDAO --> LoginService: Usuario
LoginService --> LoginServlet: Usuario
LoginServlet -> HttpSession: setAttribute("usuario", Usuario)
LoginServlet --> bienvenida.jsp: sendRedirect("bienvenida.jsp")
bienvenida.jsp --> Usuario: Muestra datos del usuario
```

5.2. Gestión de Sesiones

El sistema utiliza HttpSession para mantener la identidad del usuario entre peticiones:

```
HttpSession session = request.getSession();
session.setAttribute("usuario", usuarioLogueado);
session.setAttribute("rol", usuarioLogueado.getRol());
```

En bienvenida.jsp, se verifica que la sesión exista antes de mostrar contenido:

```
HttpSession sesion = request.getSession(false);
if (sesion == null || sesion.getAttribute("usuario") == null) {
   response.sendRedirect("login.jsp");
   return;
}
```

6. Guía de Desarrollo

6.1. Añadir un Nuevo Usuario

Para añadir un nuevo usuario al sistema, edite el constructor de UsuarioDAO.java:

```
public UsuarioDAO() {
   baseDeDatos = new ArrayList<>();
   baseDeDatos.add(new Usuario("admin", "1234", "Administrador", "ADMIN"));
   baseDeDatos.add(new Usuario("usuario", "pass", "Usuario Normal", "USER"));
   baseDeDatos.add(new Usuario("profesor", "profe123", "Profesor", "TEACHER"));

   // Añada aquí nuevos usuarios:
   baseDeDatos.add(new Usuario("nuevo_user", "pass123", "Nuevo Usuario", "USER"));
}
```

6.2. Migrar a una Base de Datos Real (JDBC)

Para migrar de la simulación en memoria a una base de datos real (MySQL, PostgreSQL, etc.):

Paso 1: Añadir la dependencia en `pom.xml`

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
```

Paso 2: Modificar `UsuarioDAO.java`

```
public class UsuarioDAO {
  private static final String URL = "jdbc:mysql://localhost:3306/gestion?useSSL=false";
  private static final String USER = "root";
  private static final String PASS = "password";

  public boolean validarCredenciales(String username, String password) {
    try {
      Class.forName("com.mysql.cj.jdbc.Driver");
      Connection conn = DriverManager.getConnection(URL, USER, PASS);
      PreparedStatement ps = conn.prepareStatement(
        "SELECT COUNT(*) FROM usuarios WHERE username=? AND password=?"
      );
      ps.setString(1, username);
      ps.setString(2, password);
      ResultSet rs = ps.executeQuery();
      rs.next();
      boolean valido = rs.getInt(1) > 0;
      conn.close();
      return valido;
    } catch (Exception e) {
      e.printStackTrace();
      return false;
    }
  }
}
```

Paso 3: Crear la tabla en MySQL

```
CREATE DATABASE gestion;
USE gestion;

CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(50) NOT NULL,
  nombre_completo VARCHAR(100),
```

```
  rol VARCHAR(20)
);

INSERT INTO usuarios (username, password, nombre_completo, rol) VALUES
('admin', '1234', 'Administrador del Sistema', 'ADMIN'),
('usuario', 'pass', 'Usuario Normal', 'USER'),
('profesor', 'profe123', 'Profesor de UF2405', 'TEACHER');
```

6.3. Añadir una Nueva Página JSP

Cree el archivo .jsp en src/main/webapp/.

Añada la directiva de página al inicio:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
```

Importe las clases necesarias:

```
<%@ page import="com.empresa.login.model.Usuario" %>
```

Acceda a los atributos de sesión:

```
<% Usuario usuario = (Usuario) session.getAttribute("usuario"); %>
```

6.4. Modificar los Estilos CSS

El archivo src/main/webapp/css/estilos.css contiene todos los estilos. Los cambios se aplican automáticamente al recargar la página (no requiere recompilación Java).

Variables de color principales:

Primario: #667eea (azul-violeta)

Secundario: #764ba2 (violeta)

Error: #c62828 (rojo)

Éxito: #28a745 (verde)

7. Patrones de Diseño Aplicados

7.1. Patrón MVC (Modelo-Vista-Controlador)

Modelo: Usuario.java (JavaBean)

Vista: login.jsp, bienvenida.jsp, error.jsp

Controlador: LoginServlet.java

7.2. Patrón DAO (Data Access Object)

Separa la lógica de acceso a datos de la lógica de negocio:

Interfaz implícita: UsuarioDAO

Implementación: Simulación en memoria (preparada para JDBC)

7.3. Patrón Service Layer

LoginService.java actúa como capa intermedia que:

Valida datos de entrada.

Aplica reglas de negocio.

Delega al DAO el acceso a datos.

8. Despliegue en Producción

8.1. Generar el Archivo WAR

Para generar un archivo .war desplegable en cualquier servidor Tomcat:

mvn clean package

El archivo se generará en: target/proyecto-login.war

8.2. Desplegar en Tomcat Externo

Copie target/proyecto-login.war en la carpeta webapps/ de su instalación de Tomcat.

Inicie Tomcat.

Acceda a: http://localhost:8080/proyecto-login/

8.3. Despliegue con Docker (Alternativa Aislada)

El proyecto incluye archivos para despliegue con Docker:

Dockerfile: Define la imagen con Tomcat y la aplicación.

docker-compose.yml: Orquesta la aplicación y una base de datos MySQL.

iniciar_todo.bat: Levanta el entorno completo.

detener_todo.bat: Detiene y limpia el entorno.

9. Seguridad

9.1. Consideraciones Actuales

Las contraseñas se almacenan en **texto plano** (solo para fines educativos).

No se implementa **protección contra fuerza bruta**.

No se utiliza **HTTPS** en el entorno de desarrollo.

9.2. Mejoras Recomendadas para Producción

Hash de contraseñas: Utilizar BCrypt o SHA-256 con salt.

HTTPS: Configurar certificados SSL/TLS.

Protección CSRF: Implementar tokens anti-CSRF.

Rate Limiting: Limitar intentos de login por IP.

Logs de auditoría: Registrar intentos de login exitosos y fallidos.

Invalidación de sesión: Llamar a session.invalidate() al cerrar sesión.

10. Troubleshooting para Desarrolladores

10.1. Error: `BUILD FAILURE - cannot find symbol`

Causa: Archivos .java desincronizados o mal ubicados.

Solución:

Verifique que la estructura de paquetes coincida con las declaraciones package.

Asegúrese de que todos los archivos estén en src/main/java/com/empresa/login/.

10.2. Error: `HTTP 404 - /proyecto-login/`

Causa: El welcome-file no está configurado o el JSP está en WEB-INF/.

Solución:

Verifique que login.jsp esté en src/main/webapp/ (NO en WEB-INF/).

Compruebe que web.xml tenga:

```
<welcome-file-list>
  <welcome-file>login.jsp</welcome-file>
</welcome-file-list>
```

10.3. Error: `HTTP 500 - Unable to compile class for JSP`

Causa: Métodos getter faltantes en Usuario.java.

Solución:

Asegúrese de que Usuario.java tenga todos los getters: getUsername(), getPassword(), getNombreCompleto(), getRol().

10.4. El servidor no arranca (puerto 8080 ocupado)

Causa: Otro proceso está usando el puerto 8080.

Solución:

Cambie el puerto en pom.xml:

```
<configuration>
  <port>8081</port>
  <path>/proyecto-login</path>
</configuration>
```

O detenga el proceso que usa el puerto 8080:

```
netstat -ano | findstr :8080
taskkill /PID <PID> /F
```

11. Referencias y Bibliografía

UF2405 - Modelo de programación web y bases de datos (Certificado de Profesionalidad IFCD0112)

Java Servlet Specification 3.1 - Oracle

JSP Specification 2.3 - Oracle

Apache Maven Documentation - https://maven.apache.org/

Apache Tomcat 7 Documentation - https://tomcat.apache.org/tomcat-7.0-doc/

Patrón MVC - Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns*.

12. Historial de Cambios

Versión	Fecha	Autor	Descripción
1.0	15/10/2026	Dev Team	Versión inicial del documento

Documento generado automáticamente para el proyecto UF2405 - UD2: Programación en Capas.

Para consultas técnicas, contacte con el equipo de desarrollo.